

HelixQA

HelixQA

Антиблефовая оркестровка QA — автономные кросс-платформенные сессии, где каждое ПРОХОЖДЕНИЕ сопровождается доказательствами, подтверждающими, что реальный пользователь может работать с функцией.

Краткое описание

HelixQA — это антиблефовый фреймворк оркестровки QA для кросс-платформенного тестирования (Android, Android TV, Web, Desktop), который объединяет банки тестов YAML, обнаружение сбоев в реальном времени, пошаговый сбор доказательств и автономные QA-сессии LLM с использованием компьютерного зрения для подтверждения работоспособности функций от начала до конца. Является обязательным типом тестирования QA согласно Constitution (§11.4.169).

Краткая характеристика

Антиблефовый оркестратор QA (Go), который запускает написанные банки тестов и полностью автономные QA-сессии, управляемые LLM и компьютерным зрением, на разных платформах — выявляет сбои, проверяет каждый шаг на соответствие собранным доказательствам (скриншоты, логи Logcat, видео, трейсы стека) и автоматически генерирует детализированные тикеты для конвейеров исправлений AI.

Подробное описание

HelixQA — это фреймворк Go, чей единственный и бескомпромиссный принцип проектирования основан на §11.4 Оперативного правила Constitution: планка для выпуска продукта определяется не тем, что «тесты прошли», а тем, что «пользователи могут работать с функцией», поэтому каждое ПРОХОЖДЕНИЕ, которое он фиксирует, должно

сопровождаться доказательствами, собранными во время выполнения — без доказательств нет зелёного света, без исключений. Он работает в двух взаимодополняющих режимах, которые вместе покрывают как сценарии по сценарию, так и неизвестные случаи.

Во-первых, **написанные банки тестов** — наборы YAML из тест-кейсов TC-XXX с указанием целевой платформы, приоритетов, упорядоченных шагов (название/действие/ожидаемый результат), тегов и ссылок на документацию — выполняются с пошаговой валидацией, обнаружением сбоев и ANR в реальном времени (ADB для Android, мониторинг процессов для веб и десктоп), централизованным сбором доказательств и автоматически генерируемыми тикетами в формате Markdown, уже подготовленными для конвейеров исправлений AI.

Во-вторых, **полностью автономная QA-сессия**, в рамках которой приложение передаётся агентам на базе LLM и компьютерному зрению, позволяя им тестировать его без участия человека в четырёх чётко структурированных фазах: подготовка (выбор LLM-моделей, построение карты функций на основе проектной документации, запуск агентов CLI, инициализация движка компьютерного зрения), проверка по документации, охватывающая все описанные функции, исследовательское тестирование с целенаправленным поиском граничных случаев и недокументированного поведения, а затем формирование отчёта и очистка с сохранением всех находок в Markdown/HTML/JSON с привязкой каждого обнаруженного дефекта к видеодоказательствам с временными метками.

Ключевой момент: фреймворк не ставит себе оценки сам. Он интегрирует четыре внешних подмодуля Go (LLMsVerifier, LLMOrchestrator, VisionEngine, DocProcessor) и использует общую инфраструктуру challenges и containers, поэтому компонент, управляющий приложением, не является тем же компонентом, который оценивает успешность его работы. Собственный набор тестов фреймворка проходит ту же самую проверку, что и тестируемые продукты, через команду `make anti-bluff` (статический анализ + манифест поведенческих якорей + мутационный ратчет) и восьмифазный оркестратор Challenge с встроенной §1.1 мутацией. Матрица покрытия типов тестов из 15 строк привязывает каждое заявленное свойство к конкретному исполняемому активу и определённой форме собранных доказательств — таким образом, утверждения фреймворка о себе самом подкреплены доказательствами в той же мере, что и вердикты, которые он выносит тестируемым продуктам.

Почему мы его создали

Традиционный QA ставит зелёный свет по принципу «утверждение прошло», и именно так в классе ошибок Constitution проскакивает *блеф* — функция, которая считается работающей, хотя для реального пользователя она сломана. HelixQA был создан специально для того, чтобы исключить такую возможность в QA: он отказывается засчитывать УСПЕХ без физических доказательств (скриншот, логкат, видео, стек-трейс, отчёт), зафиксированных в реальном исполнении, а зелёную строку сводки без таких доказательств расценивает как критический дефект, равнозначный отсутствующей функции. Кроме того, он решает проблему трудозатрат — всеобъемлющее ручное тестирование на множестве платформ не масштабируется — за счёт полной автономности сессий.

Почему это меняет правила игры

Он объединяет две вещи, которые почти никогда не сосуществуют в одном инструменте: строгий, основанный на доказательствах контроль качества и автономное, самоуправляемое исследование. Агент LLM с поддержкой компьютерного зрения открывает *реальное* приложение, проверяет каждую задокументированную функцию, выискивает недокументированные баги, на которые никто не написал тест, и формирует доказательную базу судебного уровня — так что фраза «мы это протестировали» сменяется на «вот видео, вот логкат, вот тикет». А поскольку это подмодуль QA, названный в честь Constitution, его внедрение не просто повышает честность тестирования для одной команды — оно поднимает планку качества для каждого продукта в семействе одним махом.

Что здесь инновационного

- **Контракт на доказательства против блефа** — каждый успешный результат проверки привязан к зафиксированным в реальном времени доказательствам; зелёная строка в CI считается необходимой, но никогда — достаточной, а зелёная сводка без доказательств расценивается как критический дефект.
- **Автономное исследование по документации и любопытству** — проверяет каждую задокументированную функцию, а затем отклоняется от сценария, тестируя пограничные случаи, с которыми сталкиваются реальные пользователи (пустые вводы, быстрые взаимодействия, недокументированные пути), которые не предусмотрел ни один написанный вручную набор тестов.

- **Визуальный оракул** — механическое зрение GoCV в сочетании с LLM Vision API буквально *видит* работающий интерфейс на экране, выявляя визуально нарушенные состояния, которые пропускают проверки на уровне токенов и свойств.
- **Тестовые банки на основе структуры, а не текста** — строки банка описывают структуру и генерируют подсказки для вопросов LLM в реальном времени (CONST-046), так что один банк работает для всех локалей, вместо того чтобы рассыпаться при переводе текста интерфейса.
- **Тикеты для конвейеров исправления AI** — автоматически сформированные Markdown-задачи поступают с полным пакетом доказательств, готовые к передаче напрямую агенту исправления, минуя человека-триажера.

Как это используется во всех продуктах (какие возможности даёт)

Как **обязательный столп качества** (Constitution §11.4.169 называет подмодуль `helix_qa` одним из обязательных типов тестирования), HelixQA наделяет каждый продукт в семействе одинаковым набором возможностей:

- **Автономные сессии QA:** одна команда `helixqa autonomous --project ... --platforms android,desktop,web` запускает агента LLM с поддержкой компьютерного зрения, который в автономном режиме тестирует реальные приложения, стремясь к целевому покрытию, и генерирует отчёты, тикеты и видео без участия человека.
- **Тестовые банки / наборы:** банки YAML (пороговое значение ≥ 30 на этапе `round-219`), ориентированные на платформы, с приоритетами и возможностью построчной трассировки к документации, которую они проверяют.
- **Зафиксированные доказательства:** скриншоты, логкаты, видео, стек-трейсы и полная временная шкала — централизованные и привязанные к каждому отчёту, так что любой вердикт можно воспроизвести и проверить постфактум.
- **Независимые вердикты (§11.4.141 принцип независимости):** его `issuedetector` на базе LLM и визуальный оракул оценивают поведение работающего приложения независимо от агента, который им управлял, структурно исключая классическую ошибку, когда система сама признаёт свою работу корректной.
- **Шлюз + мутационный храповик:** `make qa-all / make anti-bluff` и `challenges/scripts/helixqa_orchestrator_challenge.sh` (8 фаз, встроенная мутация §1.1) непрерывно подтверждают честность самого HelixQA — причём намеренно отсутствуют

лазейка --skip-helixqa, позволяющая отключить дисциплину под давлением дедлайнов.

Крупнейшие технические вызовы и способы их решения

- **Предотвращение ложных срабатываний в самом QA** — инструмент, выявляющий обман, не должен сам превратиться в обман → каждый этап проверяется на основе собранных доказательств, прохождение теста без доказательств засчитывается как дефект, а не как успех, а манифест привязки поведения связывает каждую заявленную возможность с исполняемым тестом (CONST-035), так что ни одна возможность не может быть заявлена без подтверждающего её теста.
- **Управление разнородными платформами из единого ядра** — Android, Android TV, Web и Desktop не имеют общей модели ввода → единый пакет navigator абстрагирует платформо-зависимые ActionExecutors (ADB, Playwright, X11) и детекторы падений для каждой платформы (android/web/desktop), так что логика оркестрации пишется один раз, а различия между платформами остаются на периферии.
- **Создание автономных агентов, полезных, а не хаотичных** — неуправляемый LLM, выпущенный в приложение, может блуждать бесконечно → LLMsVerifier оценивает и выбирает подходящие модели, LLMOrchestrator управляет безголовыми агентами CLI (opencode, claude-code, gemini, junie, qwen-code), DocProcessor строит карту функций, задающую цель исследования, а VisionEngine привязывает каждое решение к реальным пикселям на экране, а не к воображению модели.
- **Локализационно-безопасные банки тестов** — набор тестов, жёстко привязанный к английскому тексту интерфейса, ломается в пятнадцати языках → банки описывают только структуру, а пользовательский текст подсказок загружается LLM/ресурсами во время выполнения (CONST-046), так что один и тот же банк проверяет одно и то же поведение независимо от локали.
- **Доказательство, что контрольные точки не фикция** — антиобманная контрольная точка, которая сама не может упасть, — это высший обман → парные мутации §1.1 лишают тип сбора доказательств или антиобманной проверки и требуют, чтобы контрольная точка падала, а храповой механизм мутаций не даёт этой гарантии незаметно ослабнуть со временем.

Технологический стек

- **Оркестратор Go 1.24+** — *почему*: QA должен работать везде, где работают продукты, поэтому единый статически скомпонованный, быстрый и переносимый бинарный файл лучше тяжёлого на runtime решения; *как*: один CLI cmd/helixqa, предоставляющий композитные подкоманды run / list / report / autonomous / version.
- **Банки тестов YAML (pkg/testbank)** — *почему*: наборы тестов должны быть декларативными и читаемыми, редактируемыми людьми без необходимости трогать Go; *как*: version/name/test_cases[] с id, category, priority, platforms, упорядоченными steps[] и documentation_refs[] для отслеживаемости до документации функций.
- **Детекторы падений/ANR (pkg/detector)** — *почему*: самые критичные сбои происходят в реальном времени, во время взаимодействия, а не в постфактумной проверке; *как*: ADB (pidof/logcat/screencap) для Android и pgrep для web/desktop, отслеживание процесса во время выполнения теста.
- **Сбор доказательств (pkg/evidence, pkg/session)** — *почему*: антиобманный контракт реален только тогда, когда каждое прохождение теста подкреплено физическими доказательствами; *как*: скриншоты, logcat, видео и трейсы стека записываются в таймлайн SessionRecorder, на который ссылается каждый отчёт.
- **Автономная сессия (pkg/autonomous, pkg/navigator, pkg/issuedetector)** — *почему*: ручное комплексное QA на четырёх платформах не масштабируется, поэтому исследование должно быть самоуправляемым; *как*: четырёхфазный SessionCoordinator плюс ActionExecutors (ADB/Playwright/X11) и детекция багов LLM, охватывающая визуальные, UX-, доступностные и функциональные дефекты.
- **Внешние субмодули** — *почему*: повторное использование и развязка (CONST-051), а также — критически важно — отделение навигатора от оценщика; *как*: LLMsVerifier (оценка моделей), LLMOrchestrator (безголовые агенты CLI), VisionEngine (GoCV + LLM Vision), DocProcessor (карта функций/покрытие), каждый из которых является независимо управляемым компонентом.
- **Антиобманные контрольные точки + храповой механизм мутаций** — *почему*: чтобы удерживать HelixQA в рамках точного §1.1 соглашения, которое он навязывает всему остальному; *как*: сканирование make anti-bluff плюс манифест привязки поведения и храповой механизм мутаций, с helixqa_orchestrator_challenge.sh в качестве восьмифазного сквозного валидатора.
- **Матрица покрытия на 15 строк (docs/test-coverage.md)** — *почему*: CONST-050(B) требует закрытого, полностью учтённого набора типов тестов без пробелов; *как*: каждая строка привязана к конкретному исполняемому активу и

определённой форме собранных доказательств, так что покрытие — это проверенный факт, а не просто утверждение.

Заметки о статусе и честности

- **Статус: бета-версия.** Активно разрабатывается (баннер статуса README, раунд 219). Соответствует собственным стандартам антиблефа.
- **Лицензия: Apache-2.0.** Установка: `go install digital.vasic.helixqa/cmd/helixqa@latest`.

Приоритетный уровень: Helix-основной — обязательный критерий качества и антиблефа, определяющий, как семейство Helix проверяет подлинную работоспособность функций.